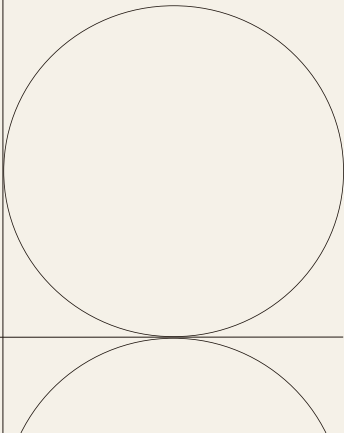


Research Paper Documentation



Overview

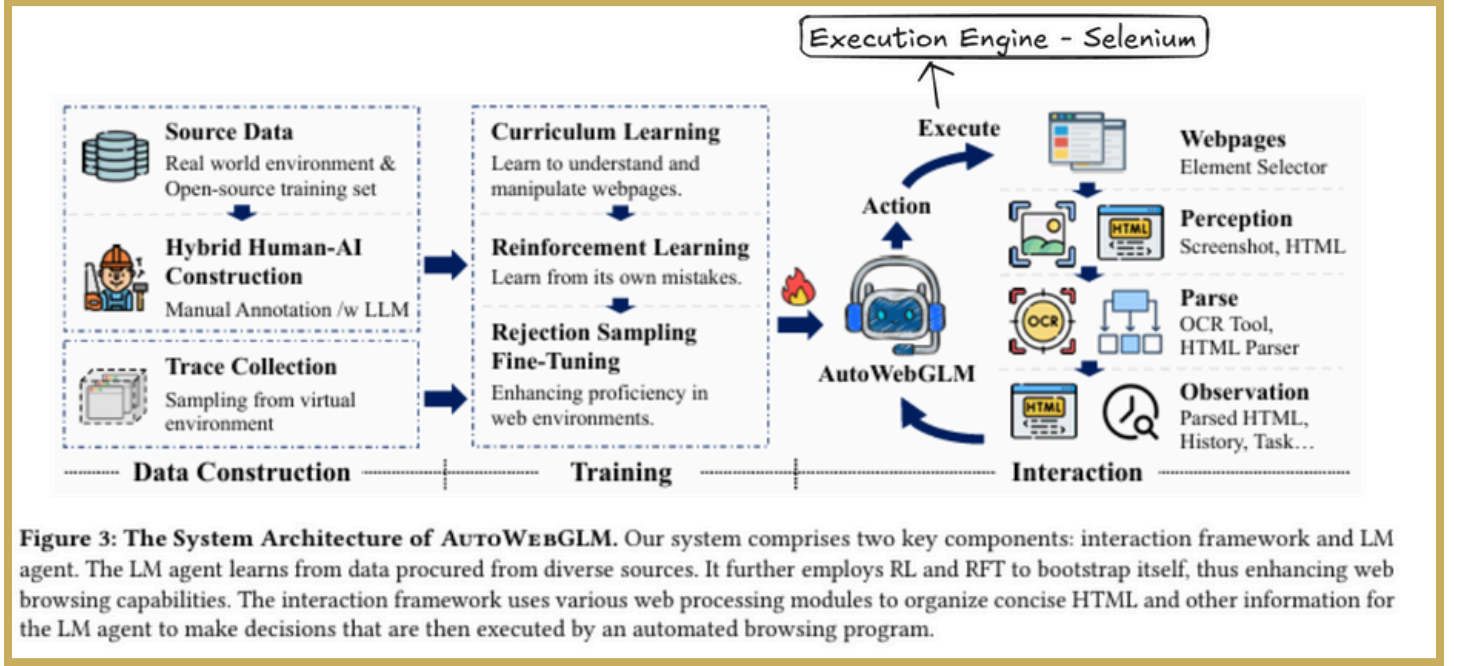
<i>Paper Name</i>	AutoWebGLM: A Large Language Model-based Web Navigating Agent
<i>Overview</i>	AutoWebGLM is a LLM based autonomous web navigation agent, designed to perform tasks like searching, form filling and various multi-step tasks. It addresses three major bottlenecks in web agents: 1. HTML complexity and verbosity 2. Absence of a universal action space 3. Lack of capability for correct inference and self-checking AutoWebGLM combines HTML simplification, curriculum learning, reinforcement learning (DPO), and rejection sampling fine-tuning (RFT) to produce a deployable agent that outperforms even GPT-4 in several web navigation benchmarks, while being significantly smaller (6B parameters).
<i>Objectives</i>	Through this report, we will explore • Problem Formulation • Architecture of AutoWebGLM • Training Procedure • Relevance to our project - AutoDevX

Problem Formulation

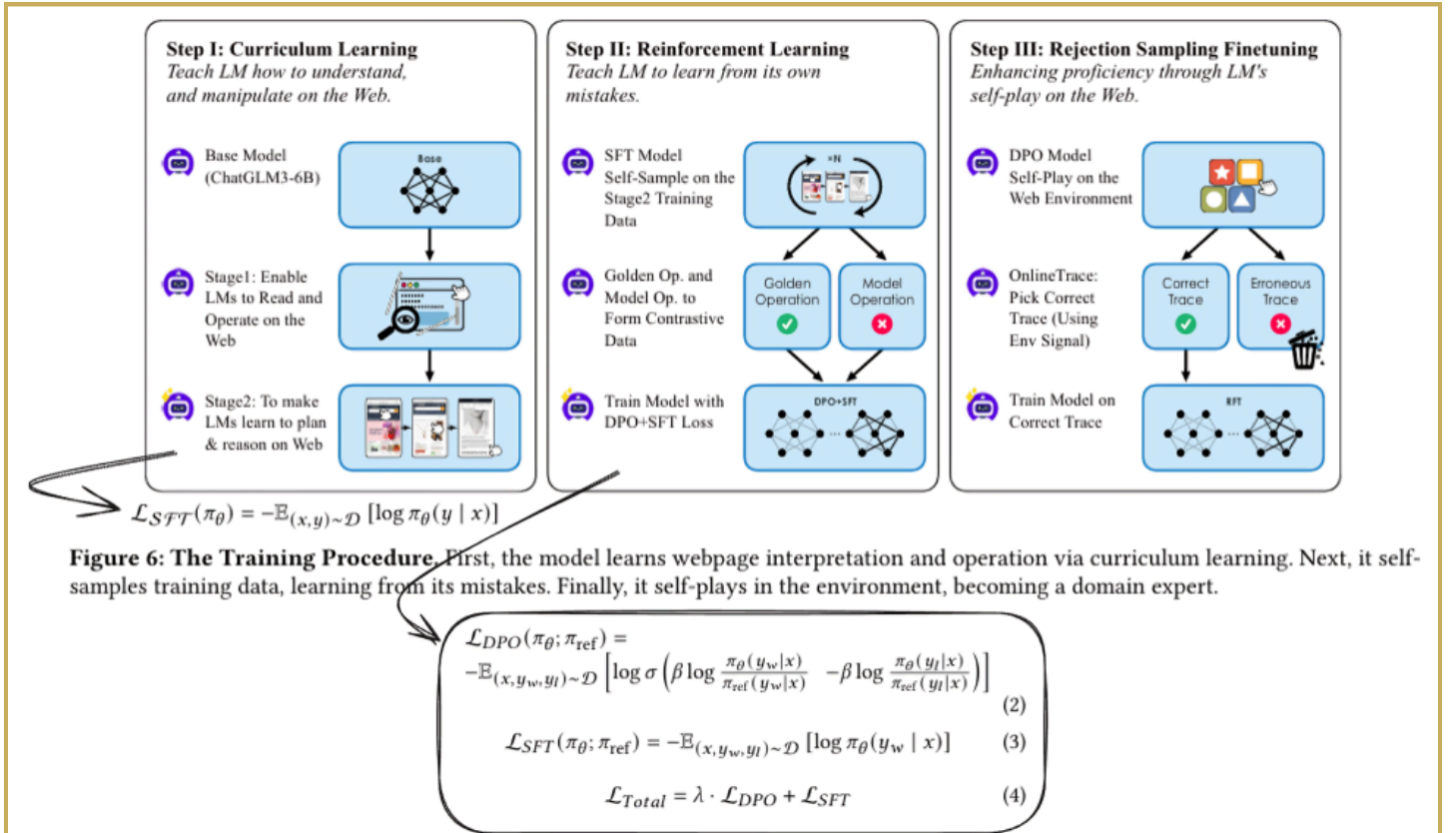
AutoWebGLM consider web browsing tasks as a sequential decision-making process.

<i>State(S)</i>	State includes the current page status, such as HTML, URL, and Window Position. $\mathbf{S} = \{\text{HTML, URL, Window Position}\}$																						
<i>Action set(A)</i>	The action set A contains all potential browsing operations, including click, type, scroll, etc. $\mathbf{A} = \{\text{click, scroll, type, . . .}\}$ Table 1: Action space for AUTOWEBGLM to interact through. <table><tr><th>Instruction</th><th>Description</th></tr><tr><td>click(id)</td><td>Click at an element</td></tr><tr><td>hover(id)</td><td>Hover on an element</td></tr><tr><td>select(id, option)</td><td>Select option in an element</td></tr><tr><td>type_string(id, text, enter)</td><td>Type to an element</td></tr><tr><td>scroll_page(direction)</td><td>Scroll up or down of the page</td></tr><tr><td>go(direction)</td><td>Go forward or backward of the page</td></tr><tr><td>jump_to(url, newtab)</td><td>Jump to URL</td></tr><tr><td>switch_tab(id)</td><td>Switch to i-th tab</td></tr><tr><td>user_input(message)</td><td>Notify user to interact</td></tr><tr><td>finish(answer)</td><td>Stop with answer</td></tr></table>	Instruction	Description	click(id)	Click at an element	hover(id)	Hover on an element	select(id, option)	Select option in an element	type_string(id, text, enter)	Type to an element	scroll_page(direction)	Scroll up or down of the page	go(direction)	Go forward or backward of the page	jump_to(url, newtab)	Jump to URL	switch_tab(id)	Switch to i-th tab	user_input(message)	Notify user to interact	finish(answer)	Stop with answer
Instruction	Description																						
click(id)	Click at an element																						
hover(id)	Hover on an element																						
select(id, option)	Select option in an element																						
type_string(id, text, enter)	Type to an element																						
scroll_page(direction)	Scroll up or down of the page																						
go(direction)	Go forward or backward of the page																						
jump_to(url, newtab)	Jump to URL																						
switch_tab(id)	Switch to i-th tab																						
user_input(message)	Notify user to interact																						
finish(answer)	Stop with answer																						
<i>History(H_t)</i>	The function ϕ updates the historical information based on the previous history H_{t-1}, the most recent action A_{t-1} and the current state S_t $H_t = \phi(H_{t-1}, A_{t-1}, S_t)$																						
<i>Policy (π)</i>	The policy π is the process for the agent to choose actions based on the current state and the history. $A_t = \pi(S_t H_t)$																						
<i>Transition Function</i>	A complete decision process starts from the initial state S_0 and history H_0, iterating through the policy π and transition function T. This iteration ceases when the action A_t is finish or reaches the maximum length. $(S_{t+1}, H_{t+1}) = (T(S_t, A_t), \phi(H_t, A_t, S_{t+1}))$																						

System Architecture



Training Procedure



Relevance to our Project

With AutoDevX, we plan to build a fully agentic, self-healing ecosystem where the agents will explore applications just like real users and will detect issues, propose fixes, validate them and continuously improve the codebase with minimal human intervention.

1. <i>HTML Simplification Algorithm</i>	AutoWebGLM's HTML pruning technique is highly relevant for our synthetic user simulation We can adapt their tree-pruning algorithm to help our agents identify clickable elements, forms, and navigation patterns efficiently and at the same time reducing token usage.
2. <i>Action Space design</i>	We can adapt their exhaustive action space as a blueprint for our synthetic user interactions Further we can extend this with developer options: 1. <code>inspect_element(id)</code> - for UI verification 2. <code>check_console_errors()</code> - for detecting runtime issues 3. <code>measure_load_time(page)</code> - for performance testing 4. <code>validate_accessibility(element)</code> - for compliance checks
3. <i>Training Pipeline</i>	Stage 1: Train agents to understand code structure, test syntax, and basic operations Stage 2: Let agents self-sample bug detection attempts, learn from false positives/negatives Stage 3: Deploy in staging environments, collect successful bug-fix trajectories, continuously improve
4. <i>RL without fragile reward models</i>	AutoWebGLM avoids classical RL reward modeling by using - Direct Preference Optimization (DPO) and Environment-level success signals